

EE782 Assignment 2

Shashank Vaibhav(22b1204) Ashok (22b1278)

October 2025

Repository and Demonstration Links

The source code for the project is available on GitHub at: [GitHub Repository](#)
Demonstration videos showing system setup and operation can be accessed at: [Demo Videos](#)

1 Introduction to the Problem statement

Design and implement an AI guard agent that uses a laptop's webcam, microphone, screen, and speakers to monitor your hostel room in your absence. The agent activates on a spoken command (e.g., "Guard my room"), recognizes trusted individuals (you, your roommate, your friends who frequent your room) via face recognition, and engages in an escalating spoken conversation with unrecognized individuals to deter intrusion. This project emphasizes integration of pre-trained AI models (speech, vision, language) into a cohesive system, requiring thoughtful system design and robust logic.

2 Approaching the problem with an FSM based design

We propose a Finite State Machine (FSM) based solution to model the behavior of the face verification system. The FSM consists of four primary states, each representing a distinct operational mode of the system:

- **Idle State:** The system remains inactive in this state. The camera module is turned off to conserve power and processing resources. Transition to the next state occurs when motion or a trigger event is detected.
- **Listening State** Following *Idle*, the system enters a lightweight *Listening* sub-step: the microphone runs a keyword-spotting routine to detect the activation hotword (for example, "Watch my room"). Upon successful hotword detection the FSM transitions to *Guard Mode*, enabling the camera and the face recognition pipeline.

- **Guard Mode:** In this state, the camera is turned on and continuously monitors for faces in the frame. The system captures and encodes facial data, comparing it against trusted embeddings. If an unrecognized or suspicious face is detected, the system escalates to the next state.
- **Escalation State:** This phase involves multiple sub-levels to represent increasing levels of alertness:
 - **Esc1:** Initial warning—system logs the event and continues observation.
 - **Esc2:** Secondary warning—system records footage and notifies the control unit.
 - **Esc3:** Critical alert—system prepares to trigger an alarm or take defensive action.
- **Alarm State:** In this final state, the system triggers an alarm to notify of unauthorized access or intrusion. Manual or programmed reset transitions the FSM back to the *Guard State*.

FSM Operation: The FSM begins in the *Idle State*, waiting for an activation signal such as motion detection or a scheduled wake-up. Upon activation, the system transitions to the *Guard Mode*, enabling the camera for continuous monitoring. If a face is recognized as trusted, the FSM remains in *Guard Mode*. However, detection of an unrecognized or suspicious face triggers the *Escalation State*, moving progressively from *Esc1* to *Esc3* depending on the severity and persistence of the threat. Once the highest escalation threshold is reached, the FSM enters the *Alarm State*, activating an alarm system and awaiting manual reset or automated cooldown before returning to *Guard*.

3 Flow between the files

The proposed face recognition and surveillance system consists of seven key Python modules that collaboratively enable real-time monitoring, verification, and escalation management. Each module performs a distinct role, while collectively contributing to a state-driven architecture managed by the Finite State Machine (FSM) defined in `state.py`. The overall workflow is described as follows:

- **capture_face.py:** This module is responsible for registering the system owner’s facial data. It captures multiple images of the authorized user through the webcam and computes facial embeddings, which are stored as trusted reference data for later verification.
- **camera.py:** A utility module used primarily for testing and initializing the webcam feed. It ensures that the camera is accessible and functioning correctly before live operations begin.

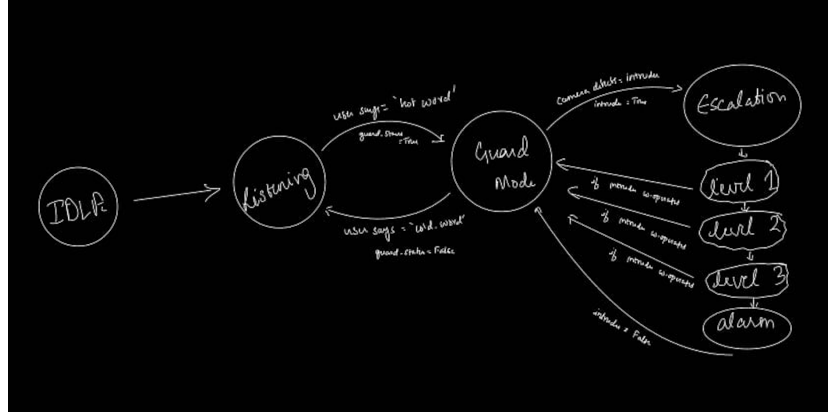


Figure 1: Finite State Machine (FSM) workflow of the proposed system.

- **face_recognition_module.py:** This module forms the core of the recognition process. It continuously processes the live video feed, detects faces in the frame, and compares their embeddings against the trusted database generated by `capture_face.py`. If a recognized face is detected, the system remains in the *Guard Mode*. However, if the face is unrecognized, it triggers the escalation mechanism.
- **llm_module.py:** When an unrecognized face is encountered, the system transitions into the *Escalation* phase. This module integrates with a Large Language Model (LLM) to generate intelligent responses or decision-making cues during suspicious activity. The responses are context-aware and dynamically adapted to the current escalation level.
- **STT.py (Speech-to-Text):** This component captures the audio input from the owner or intruder and converts it into text. It enables verbal interaction between the user and the system, allowing voice-based commands or responses to be processed during the escalation stages.
- **TTS.py (Text-to-Speech):** Complementary to the STT module, this component converts textual responses from the LLM module into spoken audio output. It provides audible feedback during escalation, allowing the system to communicate alerts or warnings verbally.
- **state.py:** This module implements the Finite State Machine (FSM) that governs the overall system behavior. It defines transitions between four major states—*Idle*, *Guard Mode*, *Escalation*, and *Alarm*. Each escalation phase (Esc1, Esc2, Esc3) is handled jointly by the LLM, STT, and TTS modules, creating a dynamic, interactive escalation process.

During operation, the system begins in the *Idle* state with the camera turned off. When activated, it transitions to *Guard Mode*, where `face_recognition_module.py`

continuously monitors for faces. A match with trusted embeddings keeps the system in Guard Mode, while detection of an unknown face triggers the *Escalation* sequence managed by the LLM, STT, and TTS modules. The escalation progresses through three levels—*Esc1*, *Esc2*, and *Esc3*—with increasingly urgent interactions. At any point, the authorized user can issue the stop command “*stop watching*,” detected via the STT module, which resets the FSM to the *Idle* state. Finally, if escalation continues without resolution, the FSM advances to the *Alarm* state, activating security alerts.

4 Testing Outcomes

The proposed system was tested extensively across multiple modules and conditions. We were able to successfully capture and store trusted facial embeddings for the owner, followed by reliable face detection with significant confidence under varying lighting conditions. The integration between the **Gemini** LLM API, TTS, and STT modules was achieved seamlessly, allowing smooth transition between text and speech during the escalation process.

We also implemented two functional hotwords: “*Watch my room*” to activate the Guard Mode, and “*Stop Watching*” to manually force the system back into Idle Mode. The model demonstrated robustness in handling lighting variations and multiple intruders (tested with 4–5 unknown individuals). Additionally, the system was able to assess the tone and mood of the speaker—such as aggression or intent to leave—using the speech parameters, which influenced the transition between escalation levels and return to Guard Mode. Overall, the system exhibited strong reliability, responsiveness, and adaptability under diverse test conditions.

5 Challenges Encountered

During the development of the face recognition and FSM-based security system, several technical challenges were encountered across different modules. The key issues and their resolutions are summarized below:

1. Speech-to-Text (STT) Module

One of the initial challenges was related to accurate speech recognition. The pre-trained speech-to-text model had difficulty understanding words due to accent variations between the training dataset and the speaker. This required the use of shorter, more clearly enunciated words to improve recognition accuracy. Another issue arose from the STT module remaining active continuously, causing unnecessary background capture and interference with other processes. To address this, the STT thread was momentarily paused during state transitions, ensuring that speech recognition only occurred when required. All STT operations were managed within a separate thread to prevent blocking of the main execution flow.

2. Text-to-Speech (TTS) Module

The TTS module initially produced an unwanted system alarm sound when triggered, particularly on macOS. This was mitigated by replacing the alarm tone with a spoken alert message. A more complex problem was the interference between the TTS and STT modules—when TTS was active, it would overlap with STT listening, leading to false transcriptions or early termination. This was solved by introducing a `tts_active` flag: when the flag was set high, STT recording was temporarily halted. Additionally, fine-tuning a small `buffer_time` between STT and TTS operations helped maintain synchronization between the two modules.

3. LLM Integration

The Large Language Model (LLM) integration was relatively straightforward. The main challenge involved ensuring proper API connectivity and managing response times during escalation. Once integrated, the LLM performed reliably across all escalation stages.

4. Main Controller (`main.py`)

Integrating all modules within `main.py` introduced several synchronization challenges. The STT loop interfered with concurrent processes, particularly the camera feed and TTS playback, as both competed for processing resources. This was resolved by disabling the camera module once an intrusion was detected, as it was unnecessary during escalation. Furthermore, TTS would sometimes terminate prematurely after the first escalation stage due to overlap with STT recording. The addition of a controlled delay (`buffer_time`) between state transitions helped stabilize the escalation sequence.

5. Camera and Environment Issues

The `camera.py` module, based on OpenCV, encountered compatibility issues when executed inside Jupyter Notebook, where live video feeds could not be rendered properly. This was resolved by migrating the execution environment to VS Code. Additionally, the `pyttsx3` library produced erratic audio outputs on macOS. Consequently, the native macOS speech synthesis command `say` (using the “Samantha” voice engine) was adopted for the TTS functionality. The `pyttsx3` library remains included in the project but could not be fully tested. Preliminary audio tests were conducted using dummy sentences to verify audio output consistency before integrating it into the FSM workflow.

Through these iterative debugging and optimization steps, the final system achieved stable multi-threaded performance, with synchronized operation between STT, TTS, LLM, and camera modules under the control of the FSM.